Python Mid-Sem Revision Notes

Introduction to Python • Control Statements • Lists • range() • Tuples

Exam-oriented concepts, syntax, code examples, important methods, differences and practice questions.

1. Introduction to Python

What is Python?

Python is a high-level, interpreted, general-purpose programming language with readable syntax. It is dynamically typed and supports object-oriented programming.

Important Features

Feature	Meaning
High-level	Easy for humans to read and write.
Interpreted	Executed by the Python interpreter.
Dynamically typed	No need to declare a variable's type explicitly.
Object-oriented	Supports classes and objects.
Portable	Can run on different operating systems with Python installed.
Open source	Freely available to use.

Variables and Data Types

```
name = "Sathish"
age = 20
marks = 85.5
passed = True

print(type(name))
print(type(age))
print(type(marks))
print(type(passed))
```

Common types: int, float, str, bool, list, tuple, set and dict.

Type Conversion

```
x = "100"
y = int(x)
z = float("10.5")
print(y, z)
```

Operators

Arithmetic: +, -, *, /, %, //, **
Comparison: >, <, >=, <=, ==, !=
Logical: and, or, not
Membership: in, not in
Identity: is, is not

```
a = 10
b = 3
print(a / b)    # division
print(a // b)   # floor division
print(a % b)    # remainder
print(a ** b)   # power
```

Input and Output

```
name = input("Enter name: ")
age = int(input("Enter age: "))
print(f"Name: {name}")
print(f"Age: {age}")
```

Remember: input() returns a string. Use int(), float(), etc. when numeric input is needed.

2. Control Statements

Control statements control the order in which statements execute. Main groups: conditional statements, loops and jump statements.

if

```
age = 20
if age >= 18:
    print("Eligible")
```

if-else

```
n = 10
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

if-elif-else

```
marks = 85
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
elif marks >= 50:
    print("C")
else:
    print("Fail")
```

Nested if

```
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible")
```

for Loop

```
for i in range(1, 6):
    print(i)
```

while Loop

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

break, continue and pass

```
# break
for i in range(1, 6):
    if i == 3:
        break
    print(i)

# continue
for i in range(1, 6):
    if i == 3:
        continue
    print(i)

# pass
if True:
    pass
```

Statement	Purpose
if / elif / else	Conditional decision making.
for	Repeats over a sequence/range.
while	Repeats while a condition is true.
break	Stops the loop immediately.
continue	Skips the current iteration.
pass	Placeholder; performs no operation.

3. Lists in Python

A list is an ordered, mutable collection. It is indexed, allows duplicate values and can contain different data types.

Create, Access and Update

```
numbers = [10, 20, 30, 40]

print(numbers[0])
print(numbers[-1])
print(numbers[1:3])

numbers[1] = 50
print(numbers)
```

Adding Elements

```
numbers = [10, 20]
numbers.append(30)
numbers.insert(1, 15)
numbers.extend([40, 50])
print(numbers)
```

Removing Elements

```
numbers = [10, 20, 30, 40]
numbers.remove(20)
numbers.pop()
print(numbers)
# numbers.clear() removes all elements
```

Important List Methods

Method	Purpose
append(x)	Adds x at the end.
insert(i,x)	Adds x at index i.
extend(x)	Adds multiple elements.
remove(x)	Removes first occurrence of x.
pop(i)	Removes and returns an element.
clear()	Removes all elements.
index(x)	Returns first index of x.
count(x)	Counts occurrences of x.
sort()	Sorts the original list.
reverse()	Reverses the original list.
copy()	Creates a shallow copy.

Built-in Functions with Lists

```
marks = [80, 35, 90, 60, 75]

print(len(marks))
print(sum(marks))
print(max(marks))
print(min(marks))
print(sorted(marks))
print(sorted(marks, reverse=True))
```

Searching and Updating

```
marks = [80, 35, 90, 60, 75]
search = int(input("Enter mark: "))

if search in marks:
    print("Found")
else:
    print("Not Found")

for i in range(len(marks)):
    if marks[i] < 40:
        marks[i] = 40
```

```
print(marks)
```

append() vs extend()

append(x) adds x as one element. **extend(iterable)** adds each element from the iterable.

```
a = [1, 2]
a.append([3, 4])
print(a)       # [1, 2, [3, 4]]

b = [1, 2]
b.extend([3, 4])
print(b)       # [1, 2, 3, 4]
```

sort() vs sorted()

sort() changes the original list. **sorted()** returns a new sorted list.

```
numbers = [30, 10, 20]
numbers.sort()
print(numbers)

numbers = [30, 10, 20]
new_numbers = sorted(numbers)
print(new_numbers)
print(numbers)
```

4. range() in Python

range() generates a sequence of integers and is commonly used with loops.

```
range(stop)
range(start, stop)
range(start, stop, step)
```

Expression	Values
range(5)	0 1 2 3 4
range(2, 6)	2 3 4 5
range(2, 10, 2)	2 4 6 8
range(10, 0, -2)	10 8 6 4 2

Important: the stop value is excluded. range(1, 6) gives 1, 2, 3, 4, 5.

```
for i in range(2, 21, 2):
    print(i)

for i in range(10, 0, -1):
    print(i)

total = 0
for i in range(1, 11):
    total += i
print(total)
```

5. Tuples in Python

A tuple is an ordered, immutable collection. It supports indexing and slicing and can contain duplicate values.

Create, Access and Slice

```
t = (10, 20, 30, 40, 50)
print(t[0])
print(t[-1])
print(t[1:4])
print(t[::-1])
```

Single-Element Tuple

```
a = (10,)   # tuple
b = (10)    # int

print(type(a))
print(type(b))
```

Immutability

```
t = (10, 20, 30)
```

```
# t[1] = 50
# TypeError: tuple cannot be changed
```

Tuple Methods

Tuples have two main methods: **count()** and **index()**.

```
t = (10, 20, 10, 30)
print(t.count(10))
print(t.index(30))
```

Packing and Unpacking

```
student = ("Rahul", 20, "CSE")

name, age, department = student

print(name)
print(age)
print(department)
```

List vs Tuple

List	Tuple
Mutable	Immutable
Uses [ ]	Uses ( )
Suitable when data changes	Suitable for fixed data
Many modification methods	Main methods: count(), index()

Built-in Functions with Tuples

```
marks = (80, 90, 70, 85, 95)

print("Total:", sum(marks))
print("Maximum:", max(marks))
print("Minimum:", min(marks))
print("Average:", sum(marks) / len(marks))
```

6. Quick Exam Revision

• **Python:** high-level, interpreted, dynamically typed.

• **Conditions:** if, if-else, if-elif-else, nested if.

• **Loops:** for and while.

• **Jump statements:** break, continue, pass.

• **List:** ordered, mutable, duplicates allowed.

• **range:** range(start, stop, step); stop is excluded.

• **Tuple:** ordered, immutable, duplicates allowed.

• **List methods:** append, insert, extend, remove, pop, clear, index, count, sort, reverse.

• **Tuple methods:** count and index.

• **Important built-ins:** len, sum, max, min, sorted, type, range.

High-Value Practice Questions

1. Explain the features of Python with examples.

2. Differentiate for and while loops.

3. Explain break, continue and pass.

4. Explain range(start, stop, step) with examples.

5. Explain list indexing, slicing and updating.

6. Explain append(), insert(), extend(), remove() and pop().

7. Differentiate append() and extend().

8. Differentiate sort() and sorted().

9. Write a program to find total, average, maximum and minimum of marks stored in a list.

10. Write a program to search for an element in a list.

11. What is a tuple? Why is it immutable?

12. Differentiate list and tuple.

13. Explain tuple count() and index().

14. Explain tuple packing and unpacking.

15. Write a program to find maximum, minimum and average of tuple elements.

Best revision method: For every concept, remember the definition, syntax, one small code example and one difference/question. Then write the larger programs without looking at the solution.